DigiT — Usage Limits & a Capacity-Gated Launch

How to cap users per plan and close new subscriptions at capacity — without shutting out existing customers · v1.0

This document designs deliberate limits on DigiT: a cap on **users per subscription plan**, and a **global capacity cap** that closes new subscriptions once you are full — while existing customers keep signing in and working normally. The framing that makes this a strength rather than a wall: it is a **capacity-gated, invite-controlled launch**. For a solo, bootstrapped founder whose costs scale with AI usage, controlling who gets in *is* the point.

> **The one design decision that makes it work:** treat sign-in and subscription as **two separate doors**. The sign-in door is **never gated** — existing customers always get in. The subscription door **closes at capacity**, sending new demand to a waitlist instead of a dead end.

1 The two limits

Two independent mechanisms — keep them distinct.

- **Per-tenant seat limit** — how many users one customer's plan allows — Starter 5, Professional 25, Enterprise custom. Checked whenever a customer tries to add a user.
- **Global capacity cap** — the total number of customers (active tenants) you are willing to serve at once. When reached, new subscriptions close and new demand goes to a waitlist. Checked whenever anyone tries to create a subscription.

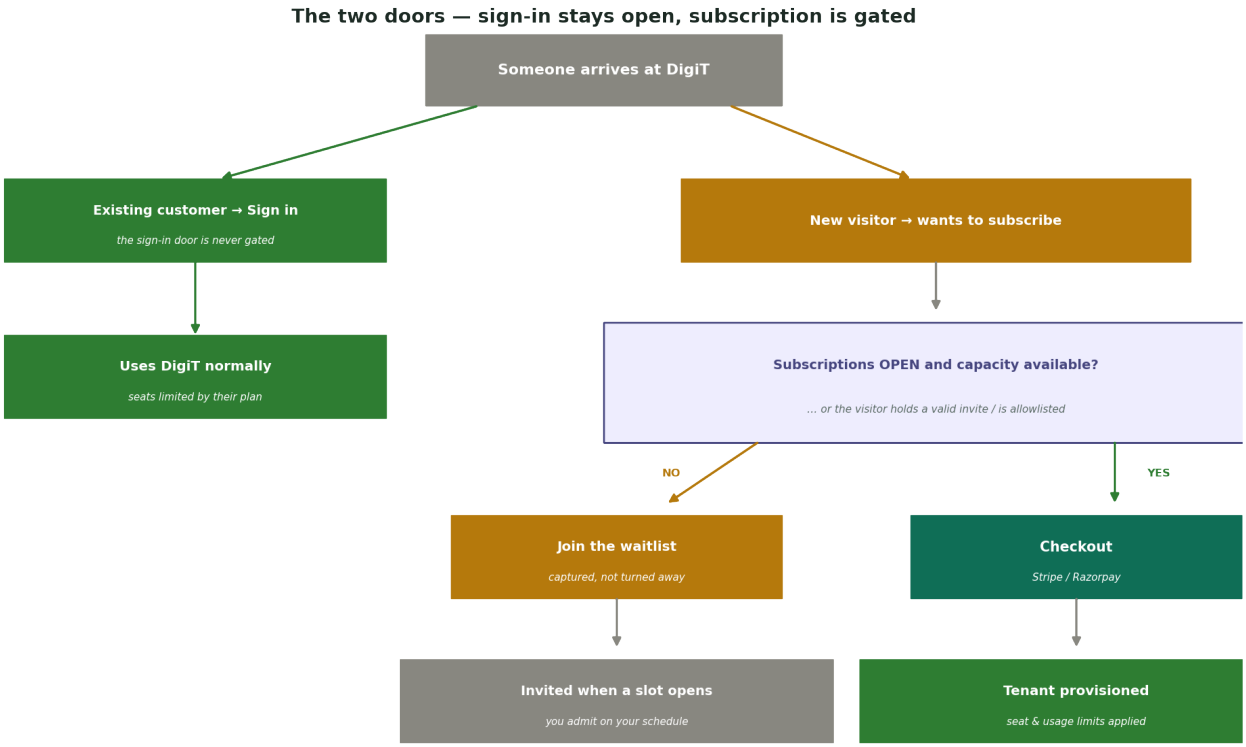

Figure 1 — The two doors. Sign-in is always open for existing customers; a new subscription must pass the capacity gate (or carry a valid invite), otherwise it becomes a waitlist entry.

2 Why this works in your favour

The case for deliberately limiting the product.

- **Cost control** — AI and infrastructure costs scale with usage. A cap stops runaway costs before your margins are proven — the single biggest reason for a bootstrapped founder.
- **Infrastructure stability** — you scale capacity on purpose, not by accident, and never overload a platform that is not yet battle-tested.
- **Quality of service & support** — solo, you can support a controlled cohort *well* — strong onboarding, fast responses, fewer fires.
- **Scarcity & exclusivity** — invite-only and a waitlist create demand signal and perceived value. “Founding customer” status becomes something people want.
- **Deliberate, staged scaling** — turn the tap gradually; learn from each cohort before widening the next.
- **Higher-quality feedback** — a small, engaged group of design partners gives far better product feedback than a flood of drive-by sign-ups.
- **Predictable economics** — a known number of customers means predictable costs and revenue — far easier to reason about pre-launch.
- **Contained risk** — fewer customers means a smaller blast radius if something breaks.
- **You can keep your promises** — you can actually meet uptime, support and SLA commitments at a capacity you chose.
- **Built-in upgrade pressure** — seat caps push growing teams to upgrade a tier — natural revenue expansion.
- **Lower abuse surface** — fewer, verified customers (tied to your onboarding checks) means less fraud and abuse to police.
- **A waitlist is an asset** — captured demand you convert on your own schedule — a ready-made pipeline, not lost visitors.
- **Focus** — fewer customers means less context-switching and more time on the product.

3 The architecture

Entitlements — the single source of truth

Every plan carries an **entitlement** record: `seat_limit`, `ai_action_limit`, `workspace_limit`, and so on. Limits are never hard-coded in the app — they are read from the plan, so changing a cap is a config change, not a code change.

Enforcement — UI is a hint, the server is the wall

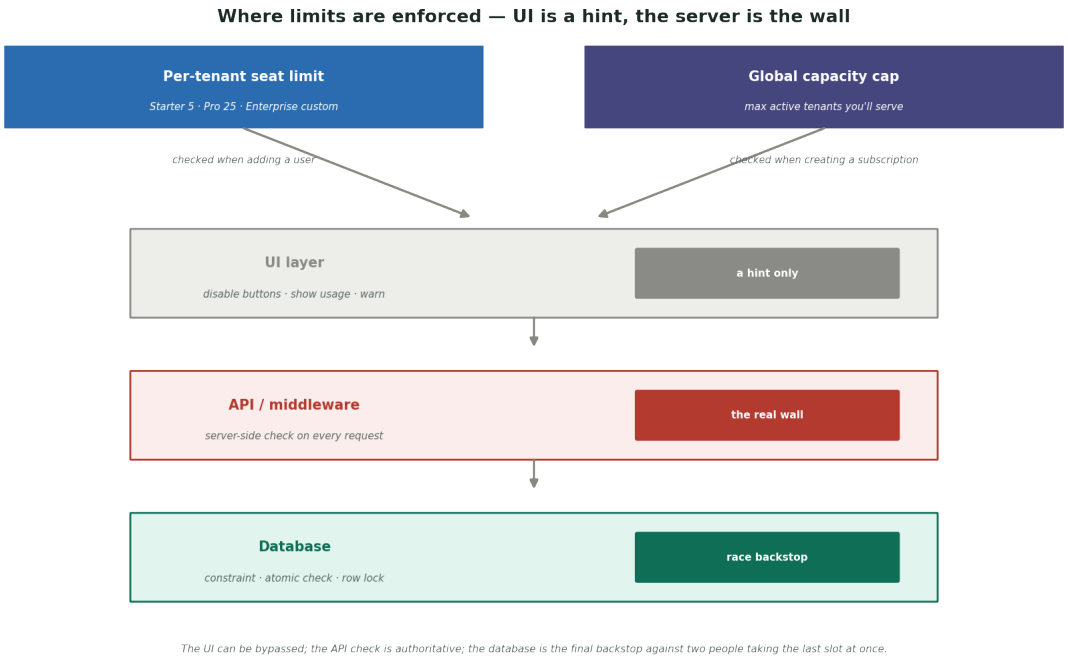

Figure 2 — Both limits are enforced through the same three layers. Only the server layer is authoritative.

- **UI layer (a hint)** — disables the “invite” or “subscribe” button, shows usage (“4 of 5 seats”), and warns. Helpful, but can be bypassed — never rely on it alone.
- **API / middleware (the real wall)** — every request to add a user or create a subscription is checked server-side against the entitlement and the capacity cap. This is the authoritative gate.
- **Database (the backstop)** — a constraint or an atomic check-and-increment inside a transaction, so two people cannot both take the last seat or the last capacity slot at the same instant (the race condition).

The subscription gate

Before creating a Stripe or Razorpay subscription, the server asks three things: is the master switch `subscriptions_open` on? Is `active_tenants < capacity_cap`? Or does the visitor hold a valid invite / sit on the allowlist? If (open and under cap) **or** invited → proceed to checkout and provision the tenant. Otherwise → route to the waitlist. Crucially, this gate sits **only** on the subscribe path — the sign-in path is untouched.

The waitlist & invite / allowlist system

- **Waitlist** — when closed, new visitors leave their email / company and join a queue (status `waiting`). They are captured, not turned away.
- **Invites & allowlist** — time-limited invite codes (or an allowlisted email / domain) let specific people through even when closed — for founding customers and sales-led deals. This is how you admit on your own terms.
- **Admission** — when a slot frees, you issue an invite to the next in line; their code bypasses the gate for a limited window.

The capacity lifecycle

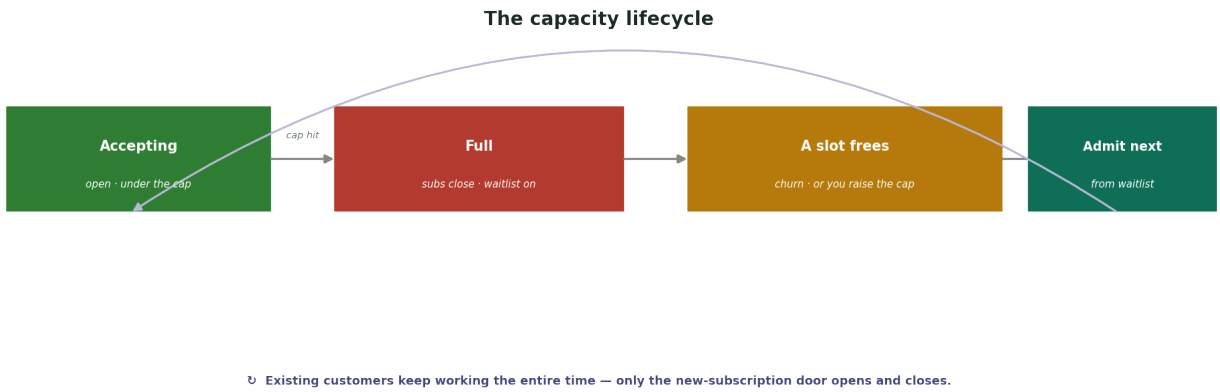

Figure 3 — Accepting → Full (subscriptions close, waitlist on) → a slot frees or you raise the cap → admit the next in line. Existing customers are unaffected throughout.

4 Control & data

Admin control (the control plane)

All of this is operated from your platform-admin layer: set the `capacity_cap`, flip `subscriptions_open` (a feature-flag / kill-switch), watch live capacity, manage the waitlist, and issue invites. The `subscriptions_open` flag doubles as an emergency brake — one switch halts all new sign-ups if costs or load spike.

The data model, in plain terms

Table	Key fields	Purpose
`plans` / `entitlements`	`seat_limit`, `ai_action_limit`, `workspace_limit`	what each plan allows
`tenants`	`plan_id`, `status`, `created_at`	each customer & their plan
`memberships`	`tenant_id`, `user_id`, `status`	count active for seat usage
`platform_config`	`subscriptions_open`, `capacity_cap`	the global switch & cap
`waitlist`	`email`, `company`, `status`, `position`	captured demand
`invites`	`code`, `email`, `tier`, `expires_at`	controlled admission

Edge cases to handle

- **Counting the cap** — count active paying tenants; decide whether trials count toward the cap (usually a small trial buffer sits on top).
- **Churn frees a slot** — when a customer cancels, capacity opens — trigger the next waitlist invite automatically.
- **Downgrades below seat use** — if a plan drops below the current user count, do not lock people out — block *adding* more and prompt to remove users or add seats.
- **The last-slot race** — the atomic database check is what stops two simultaneous checkouts from both succeeding at the final slot.
- **In-flight checkouts** — briefly reserve a slot when checkout begins so a started payment is not rejected at the finish line.

5 The user-facing states, at a glance

Who	Sign in?	Subscribe?
Existing customer	Always yes	N/A (already in)
New visitor — capacity available	—	Yes → checkout
New visitor — at capacity	—	No → waitlist
Invited / allowlisted visitor	—	Yes (bypasses the cap)

6 To make it work well

Honest cautions on top of the design.

- **Enforce server-side, always** — a UI-only limit is trivially bypassed. The API check is the limit.
- **Make “closed” graceful** — a waitlist with a clear message (“we open in cohorts”) keeps goodwill; a bare “blocked” page loses it.
- **Frame scarcity as intentional** — position it as an invite-only / founding-cohort launch, so it reads as exclusive, not broken.
- **Revisit the cap regularly** — a cap set too low starves growth and revenue — raise it as your costs and confidence allow.
- **Only works with real demand** — a waitlist is powerful when people want in; pair it with the founding-customer discount to drive sign-ups.

7 How this maps to DigiT

The pieces already exist: your **pricing tiers** define the seat limits (5 / 25 / custom); your **founding-customer discount** is the scarcity lever; the **platform-admin layer** is where you set the cap and flip the switch; and `subscriptions_open` is exactly a **feature flag / kill switch** from that admin design. This architecture wires them into one deliberate, capacity-gated launch.